TABLE I
OVERALL PERFORMANCE ON THE XBOW BENCHMARK.

System	Solve Rate	Avg. Time (s)	Model
AWE	51.9% (54/104)	53.1	Claude Sonnet 4
MAPTA	76.9% (80/104)	190.8	GPT-5

appropriate baseline for measuring the benefits of AWE's specialization-oriented design. We use MAPTA's publicly reported per-challenge results for all comparisons.

C. Model Selection

Before large-scale evaluation, we compared several modern LLMs within AWE's orchestration layer using DVWA. Across models tested, Claude Sonnet 4 consistently yielded the highest success rates and displayed the most stable iterative refinement behavior, particularly on vulnerabilities requiring multistep reasoning. The detailed numerical results appear in VI-A, where we revisit this analysis alongside full evaluation. Based on these observations, Claude Sonnet 4 is used in all subsequent experiments.

D. Experimental Configuration

AWE is evaluated in its aggressive configuration, which performs deep reconnaissance and executes all agents deemed relevant by the orchestrator. Each challenge is allotted a ten-minute time budget, matching MAPTA's configuration to ensure comparability. All experiments execute on identical hardware, and each challenge is run in an isolated environment to prevent cross-contamination. Memory state is reset between challenges to evaluate single-engagement performance, and browser-based verification is performed using a consistent, headless Chromium configuration.

E. Evaluation Metrics

We assess AWE along three principal dimensions: effectiveness, efficiency, and cost. Effectiveness is measured using overall and per-category solve rates, as well as the number of challenges uniquely solved by AWE or MAPTA. Efficiency metrics include time-to-solve and token usage per successful exploit, capturing both responsiveness and resource requirements. Cost metrics reflect total API expenditure and amortized cost per solved challenge based on provider pricing.

F. Sucess Criteria

A challenge is considered solved only if AWE retrieves the correct flag through a verified exploit. Partial progress or vulnerability detection without successful exploitation is not counted toward effectiveness metrics. This strict criterion ensures that all reported successes correspond to practically realizable attacks.

VI. EVALUATION

We evaluate AWE through a two-stage methodology. First, we conduct controlled experiments on DVWA to isolate the contribution of the underlying language model and justify our choice of Claude Sonnet 4. Second, we benchmark AWE against MAPTA, the most diverse publicly documented autonomous penetration-testing system, on the full 104-challenge XBOW benchmark.

This combination of controlled and large-scale testing provides a comprehensive view of AWE's capabilities, limitations, and efficiency.

A. Evaluation on DVWA

DVWA provides a stable, deterministic environment that enables fine-grained comparison of LLM behavior independent of broader architectural factors. We executed AWE with three LLMs - Claude Sonnet 4, GPT-4o, and Gemini 2.0 Flash using identical agent logic and verification procedures across five representative vulnerability classes. Across all models, reflected XSS served as a baseline of capability, with each model achieving 100% success. Performance diverged sharply, however, once contextual reasoning or iterative inference became essential. Claude Sonnet 4 consistently outperformed both GPT-4o and Gemini on stored XSS with CSP enforcement and blind SQL injection the two categories that require AWE's most complex reasoning loops. For CSP-enforced stored XSS, Claude and GPT-4o tied at 67% accuracy, whereas Gemini dropped to 50%. For blind SQLi, Claude reached 70%, GPT-4o 60%, and Gemini 55%. These gaps reflect model-dependent differences in temporal inference, semantic constraint handling, and multi-step payload refinement. We also examined iteration efficiency. Claude converged in 10–40 payload attempts, whereas GPT-4o required roughly 20% more attempts and Gemini nearly 40% more. Given that AWE performs many such cycles for complex vulnerability classes, convergence stability directly affects time and cost. Collectively, these DVWA results demonstrate that Claude Sonnet 4 provides the best balance of accuracy and reasoning efficiency. For this reason, all subsequent experiments use Claude Sonnet 4 as AWE's underlying model.

B. Evaluation on XBOW Benchmark

We now evaluate AWE on the XBOW benchmark, a suite of 104 containerized web challenges spanning 26 vulnerability categories, ranging from single-step injections to multi-stage exploitation workflows. We use MAPTA as a baseline because it represents the most capable peer system: it employs GPT-5 in extended-reasoning mode and executes arbitrary code within a sandbox, enabling broad exploration beyond what AWE's specialized agents support.

Overall Performance: Table I presents aggregate results. MAPTA attains a higher solve rate (76.9%) than AWE (51.9%), reflecting the advantage of its general-purpose, unrestricted sandbox execution. Despite this, AWE exhibits dramatic efficiency advantages. AWE's average solve time is 53.1 seconds, which is much faster than MAPTA's 190.8 seconds. AWE's total token usage is 1.12M compared to MAPTA's 54.9M, a 98% reduction. Correspondingly, AWE's total API cost is $7.73 versus MAPTA's $21.38, despite MAPTA running on a substantially more capable model. These efficiency

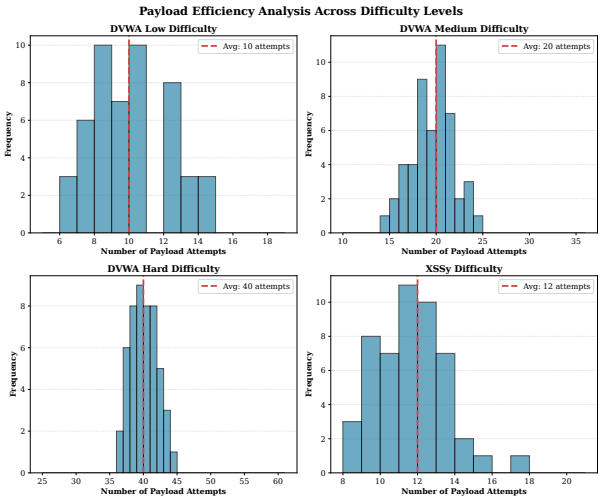

Fig. 4. Average number of payload iterations required for successful exploitation by each model. Claude Sonnet 4 converges in the fewest attempts (10–40), followed by GPT-4o with about 20% more iterations and Gemini 2.0 Flash with about 40% more. This demonstrates Claude's superior efficiency and reasoning stability.

TABLE II
COST AND TOKEN EFFICIENCY COMPARISON.

System	Total Cost	Cost/Solve	Tokens	Tokens/Solve
AWE	$7.73	$0.113	1.12M	20.7K
MAPTA	$21.38	$0.267	54.87M	685.9K

gains demonstrate that a specialization-oriented architecture can deliver orders-of-magnitude improvements in operational cost and latency.

C. Per Category Comparison

A finer-grained analysis reveals complementary strengths. Table III summarizes category-wise results for the main vulnerability classes. AWE dominates on the injection classes it explicitly targets, while MAPTA performs better on tasks requiring multi-step reasoning or semantic exploration. Notably, the two systems perform comparably on the classical injection families - SQLi, blind SQLi, and XXE - where both models reliably detect straightforward exploitation patterns.

AWE's strongest result appears in XSS. Across 23 challenges, it solves 20, substantially surpassing MAPTA's 13. The XSS cases solved exclusively by AWE typically require precise alignment between payload structure and the reflection context (e.g., attribute-versus- string contexts), adaptive filter bypassing based on observed responses, and reasoning about multi-encoding transformations. MAPTA's general-purpose reasoning pipeline often failed to infer these context-specific constraints.

AWE also performs well on blind SQL injection due to its structured inference workflow and backend-specific timing probes. Conversely, MAPTA substantially outperforms AWE in categories involving long-horizon procedural reasoning, such as privilege escalation, insecure deserialization, and business logic flaws. These tasks exceed the current capabilities of AWE's specialized-agent design.

TABLE III
CATEGORY-WISE PERFORMANCE COMPARISON ON XBOW FOR INJECTION VULNERABLITIES.

Vulnerability	Total	MAPTA		AWE	
		Count	%	Count	%
XSS	23	13	57%	20	87%
Blind SQLi	3	1	33%	2	67%
SQLi	6	6	100%	6	100%
XXE	3	3	100%	3	100%
SSRF	3	3	100%	3	100%
SSTI	13	11	85%	7	54%
Command Injection	11	9	82%	5	45%

D. Failure Modes

AWE failed on 50 challenges; MAPTA failed on 24; both systems failed on 15. Categorizing AWE's failures reveals that one-third correspond to vulnerability classes intentionally outside its scope (e.g., business logic, deserialization, cryptographic misuse). Another quarter required multi-step reasoning and stateful exploitation chains that AWE's agents currently cannot express. The remainder fall into authentication irregularities, heavy filtering that resisted AWE's mutation engine, or extremely narrow exploitation windows (e.g., race conditions).

Challenges solved only by AWE primarily fall into XSS and blind SQLi, reaffirming that its specialized exploitation pipelines provide meaningful advantages even against a more capable underlying model. Conversely, MAPTA-only solves overwhelmingly cluster in categories requiring exploration, multi-agent state management, and semantic reasoning.

E. Effeciency Analysis

AWE's primary strength is efficiency. Over the full benchmark, AWE consumed 1.12M tokens compared to MAPTA's 54.9M - an approximately 98% reduction. This efficiency stems from two architectural choices: specialized agents avoid the expansive search spaces characteristic of general-purpose reasoning, and memory-guided heuristics significantly reduce redundant attempts.

Time-to-solve exhibits a consistent 4–5× speedup across percentiles. The median solve time for AWE is 35.7 seconds compared to MAPTA's 156.2 seconds. These gains demonstrate that targeted vulnerability analysis can dramatically reduce overhead without sacrificing performance on its intended classes.

F. Summary

Our evaluation highlights a clear architectural trade-off. MAPTA achieves broader coverage due to its highly expressive sandbox and frontier-grade model, enabling multi-step exploitation across numerous vulnerability categories. AWE, in contrast, shows that architectural specialization can outperform general-purpose reasoning by large margins on targeted vulnerability classes, even when using a smaller model. The

efficiency benefits - 63% cost reduction, 4.4× faster solves, and 98% fewer tokens suggest that specialized systems may be preferable for high-frequency testing and integration into continuous assessment pipelines. At the same time, AWE and MAPTA demonstrate complementary strengths, pointing toward hybrid designs that combine structured domain knowledge with general-purpose semantic exploration.

VII. Discussion

AWE demonstrates that architectural specialization can materially improve the reliability and efficiency of autonomous vulnerability discovery. Its results highlight a broader observation about LLM-driven security testing: general-purpose reasoning alone is insufficient for precise, context-dependent exploitation, while carefully engineered task structure can compensate for smaller model capacity and dramatically reduce computational overhead.

Across XSS and blind SQLi, AWE's performance stems from explicit modeling of the execution context reflection positions, sanitization behavior, SQL operator boundaries and conditioning payload generation on these abstractions. These constraints reduce the search space an LLM must navigate and yield more stable exploit synthesis than unconstrained reasoning. That AWE outperforms MAPTA on these tasks, despite using a substantially weaker model, suggests that exploit success depends at least as much on architectural priors as on raw model capability.

At the same time, our evaluation shows that specialization does not replace broad autonomous reasoning. MAPTA's advantages are pronounced on multi-step exploitation involving authentication workflows, privilege escalation, and semantic business logic. These tasks require long-horizon planning and cross-endpoint state tracking capabilities deliberately outside AWE's design. The contrasting strengths of the two systems indicate that effective autonomous penetration testing will likely require hybrid architectures that combine structured vulnerability analysis with general-purpose exploratory reasoning.

AWE's efficiency - 98% fewer tokens, 63% lower cost, and 4.4× faster solves suggests immediate applicability in continuous or high-frequency testing settings where general-purpose agents remain prohibitively expensive. The ability to embed domain knowledge into agent design also opens the door for adaptive long-term learning: storing filter signatures, past bypasses, and effective payload patterns may enable stable performance across evolving application landscapes.

VIII. Limitations

AWE's design introduces several boundaries that shape its current applicability:

- **Scope restrictions.** The system targets injection-centric vulnerabilities and does not attempt reasoning-heavy categories such as business logic, complex authentication workflows, or protocol-level issues (e.g., request smuggling or desynchronization).
- **Limited multi-step planning.** AWE's agents operate in largely independent pipelines and do not coordinate multi-stage exploitation sequences. Tasks requiring chained discovery default credentials → IDOR → privilege escalation fall outside its reach.
- **Reliance on heuristic abstractions.** While effective, AWE's context and filter models encode assumptions about server behavior and sanitization patterns. Highly idiosyncratic frameworks or obfuscated sinks may invalidate these abstractions.
- **LLM sensitivity.** Although Claude Sonnet 4 performed best in our analysis, model-dependent reasoning variability remains a systemic constraint; shifts in model behavior or pricing may affect long-term stability.

IX. Conclusion

This work introduces AWE, a specialized multi-agent system that rethinks how LLMs can support autonomous web exploitation. By embedding domain knowledge into the architecture rather than relying solely on free-form reasoning, AWE achieves high accuracy on targeted vulnerability classes and delivers large efficiency gains over a state-of-the-art general-purpose system. The contrast with MAPTA underscores a central insight: precision exploitation benefits from structure, while broad coverage benefits from flexibility.

A natural direction forward is the integration of these paradigms combining specialized agents that capture the semantics of injection vulnerabilities with higher-level agents capable of planning multi-step attacks. Such hybrid approaches may enable autonomous penetration testing systems that are both scalable and semantically capable, bringing fully automated web security analysis closer to practical reality.

References

[1] OWASP Foundation, "OWASP Top 10." Available: https://owasp.org/Top10/. Accessed: Aug. 21, 2025.

[2] PortSwigger, "Burp Suite Web Vulnerability Scanner." Available: https://portswigger.net/burp. Accessed: Aug. 21, 2025.

[3] OWASP Foundation, "OWASP Zed Attack Proxy (ZAP)." Available: https://www.zaproxy.org/. Accessed: Aug. 21, 2025.

[4] ProjectDiscovery, "Nuclei: Fast and Customizable Vulnerability Scanner." Available: https://github.com/projectdiscovery/nuclei. Accessed: Aug. 21, 2025.

[5] D. Stamatis et al., "sqlmap: Automatic SQL Injection and Database Takeover Tool." Available: https://sqlmap.org/. Accessed: Aug. 21, 2025.

[6] Positive Technologies, "Web application vulnerabilities in 2020–2021." Available: https://global.ptsecurity.com/en/research/analytics/web-vulnerabilities-2020-2021/. Accessed: Aug. 21, 2025.

[7] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: An LLM-empowered automatic penetration testing tool," arXiv:2308.06782, 2023. Available: https://arxiv.org/abs/2308.06782.

[8] B. Wu, G. Chen, K. Chen, X. Shang, J. Han, Y. He, W. Zhang, and N. Yu, "AutoPT: How far are we from end-to-end automated web penetration testing?," arXiv:2411.01236, 2024. Available: https://arxiv.org/abs/2411.01236.

[9] J. W. Stokes, A. Swaminathan, J. Xu, G. McDonald, X. Bai, D. Marshall, S. Wang, and Z. Li, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," arXiv:2403.01038, 2024. Available: https://arxiv.org/abs/2403.01038.

[10] Q. Wang, G. Yang, J. Wang, M. Li, Z. Chang, Y. Huang, and Z. Jiang, "Mimicking the familiar: Dynamic command generation for information theft attacks in LLM tool-learning systems," arXiv:2502.11358, 2025. Available: https://arxiv.org/abs/2502.11358.

[11] V. Mayoral-Vilches, L. J. Navarrete-Lozano, M. Sanz-Gómez, L. Salas Espejo, M. Crespo-Álvarez, F. Oca-Gonzalez, F. Balassone, A. Glera-Picón, U. Ayucar-Carbajo, J. A. Ruiz-Alcalde, S. Rass, M. Pinzger, and E. Gil-Uriarte, "CAI: An open, bug bounty-ready cybersecurity AI," arXiv:2504.06017, 2025. Available: https://arxiv.org/abs/2504.06017.

[12] I. David and A. Gervais, "Multi-agent penetration testing AI for the web," arXiv:2508.20816, 2025. Available: https://arxiv.org/abs/2508.20816.

[13] R. Dewhurst, "Damn Vulnerable Web Application (DVWA)," 2025. Available: https://github.com/digininja/DVWA. Accessed: Aug. 21, 2025.

[14] XBOW Engineering, "XBOW Validation Benchmarks." Available: https://github.com/xbow-engineering/validation-benchmarks. Accessed: Dec. 1, 2024.